

Project Proposal

Project Title

Galactic Defender – Assembly Language Game

Group Members

Neeraj Khemani – 24K-0548

Muhammad Ali Naqvi – 24K-1004

1. Introduction

- **Background:** Video games are an excellent way to demonstrate core programming principles such as logic, input handling, and real-time updates. Implementing a game in Assembly language using MASM and the Irvine32 library provides valuable hands-on experience with low-level programming concepts such as cursor control, loops, procedures, and memory management.
- **Problem Statement:** Most beginner Assembly projects (e.g., calculators, number converters) lack interactivity and real-time features. This project addresses the need for a more engaging and challenging demonstration of Assembly skills by creating an interactive, real-time text-based spaceship battle game.
- **Objectives:**
 - To design and implement a text-based arcade-style shooter game in Assembly using MASM and Irvine32.
 - To demonstrate real-time player input, enemy AI movement, collision detection, and scoring.
 - To implement progressive game difficulty through multiple levels.
 - To showcase practical application of Assembly programming beyond simple academic exercises.

2. Scope of the Project

- **Inclusions:**
 - Three game levels (Beginner, Intermediate, Advanced).
 - Player spaceship movement and shooting.
 - Enemy spawning, movement, and firing.
 - Collision detection between bullets, enemies, and the player.
 - Scoring system and level progression.
- **Exclusions:**
 - Advanced graphics (limited to console-based text rendering).
 - Multiplayer functionality.
 - Sound effects beyond basic console beeps.

3. Project Description

- **Overview:** “Galactic Defender” is a text-based arcade-style shooter game built in Assembly Language using MASM and Irvine32. The player controls a spaceship positioned at the bottom of the screen, tasked with eliminating waves of enemy ships. The game progresses through three difficulty levels with increasingly faster and smarter enemies.
- **Technical Requirements:**
 - Platform: Windows
 - IDE: Visual Studio
 - Assembler: MASM (Microsoft Macro Assembler)
 - Library: Irvine32
 - Architecture: x86 compatible
- **Project Phases:**
 - **Research & Planning** – Review Irvine32 functions for console I/O, randomization, and cursor positioning.
 - **Design** – Define data structures for player, enemies, bullets, and score.
 - **Implementation** – Build core modules: player movement, shooting, enemy spawning, collisions.
 - **Testing** – Debug gameplay logic, ensure smooth transitions between levels.

4. Methodology

- **Approach:** The project will follow an incremental development model, starting with a basic version (single enemy, single bullet), then progressively adding features such as multiple enemies, collision handling, scoring, and level difficulty.
- **Team Responsibilities:**
 - Member 1:
 - Player controls (movement, shooting).
 - Bullet handling and collision detection.
 - Scoring system and level progression.
 - Member 2:
 - Enemy spawning and movement patterns.
 - Enemy bullet mechanics and AI behavior.
 - Game interface (screen updates, display formatting, game over/win messages).

5. Expected Outcomes

- **Deliverables:**
 - A fully functional console-based arcade-style shooter game in Assembly (MASM + Irvine32).
 - A short project report including design explanation, source code, and user guide.
- **Relevance:** This project is directly relevant to the Computer Organization and Assembly Language (COAL) course because it demonstrates:
 - Practical use of x86 assembly instructions and addressing modes.
 - Implementation of input/output handling, loops, and procedures using MASM and Irvine32.
 - Application of control flow, data movement, and memory management in a real-time interactive program.
 - Connection between low-level CPU operations (registers, instructions) and higher-level program behavior (game mechanics).

By integrating these COAL concepts into a playable game, the project provides both an educational and entertaining demonstration of assembly language programming.

6. Resources Needed

- **Software:**
 - Microsoft Visual Studio
 - MASM Assembler
 - Irvine32 library
- **Other Resources:**
 - Assembly programming references (textbooks, tutorials).
 - Online guides on MASM + Irvine32 usage.
 - Support from instructor for debugging and refinement.